

Gerald Weber

Transaction Processing

How to perform composite updates safely for a remotely and concurrently accessed system.

Motivation and first ACID properties

Motivating Example: Flight booking

- An online multi-leg flight booking system (web or app).
- Users submit their orders and expect them to be:
 - Processed as **quickly as possible**, also meaning that the user gets quick feedback on success/no success.
 - **Exactly** as requested, in particularly **complete**:
 - Book all legs of the flight selected, or do nothing.
- Example problem **Atomicity**:
 - My system or the server or the network can fail.
 - But I want my order done as a whole.
- The solutions are discussed in transaction processing.

Atomicity

Another Example

In a transfer transaction from account 123 to account 321:

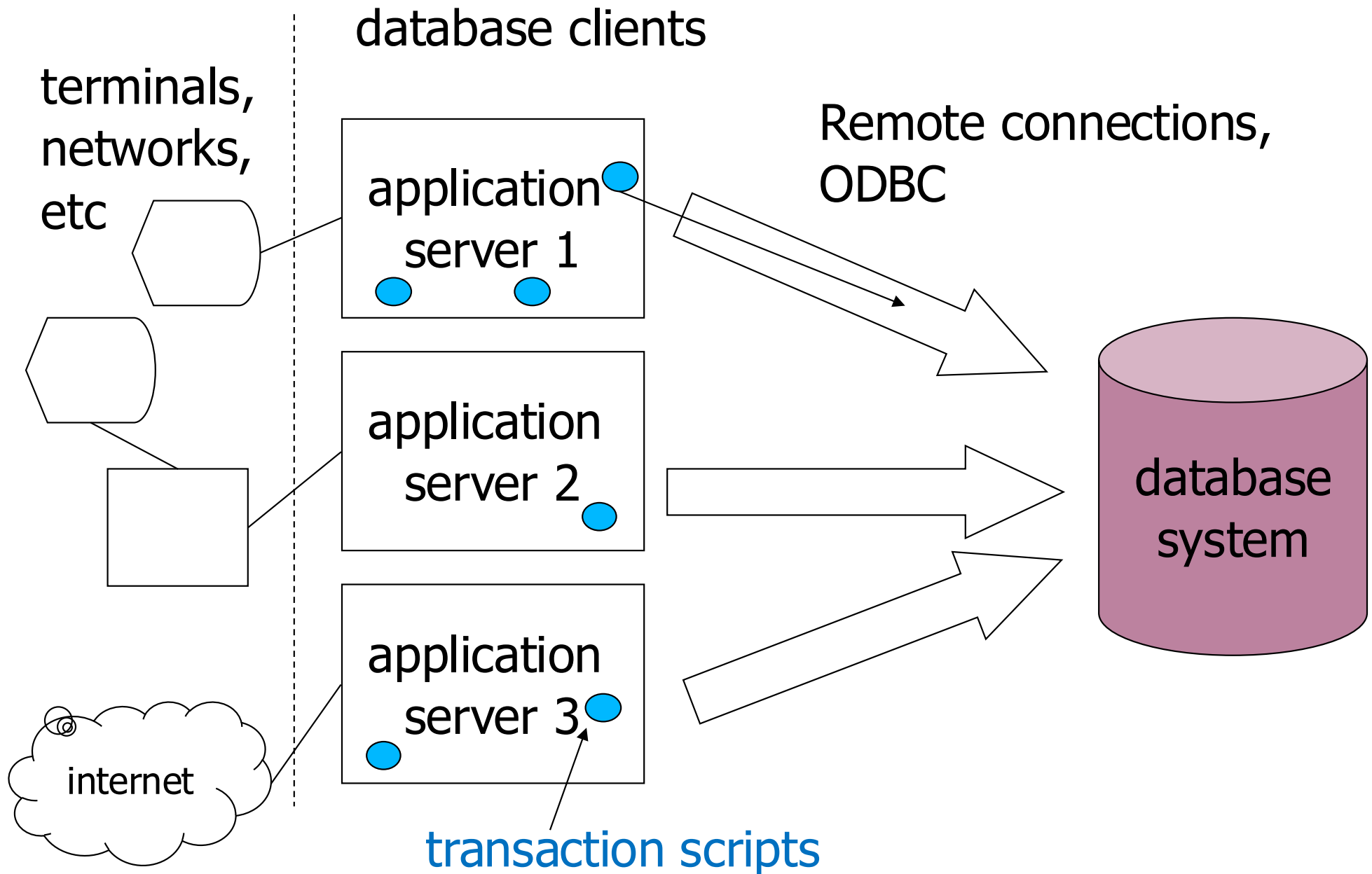
- i.) withdraw \$100 from account 123
- ii.) put \$100 on account 321

- It must not happen that the transaction stops after i.) and makes i.) persistent (durable), and never executes ii.)
- Why not? This would violate an application level consistency constraint:
 - Overall balance must be kept.

Transaction Demarcation for Atomicity

- Example problem **Atomicity**:
 - My system or the server or the network can fail.
 - But I want my order done as a whole.
- Needs **transaction demarcation**:
- System must know:
 - when does one user order (start and) **end**:
 - Operation to signal end of transaction: **commit()**.
- **Transaction**: A sequence of operations that form a logical unit; sequence of operations delimited by TA demarcation.
- **Transaction script**: a subprogram issuing a transaction if called.

Typical architecture for use of databases



Rollback for Atomicity

1. The client needs to issue a commit() request. Commit must be requested by client: Only if commit has been requested, writes in the transaction may become durable.

This protects against failures that lead to incomplete transaction processing:

Hence: **Database must take care of the rollback in case no commit has been registered.**

1. b. client can request abort as a programming feature.
2. Atomicity: Either all writes of the transaction, or none become durable: If any write becomes durable, all must become durable.

Example Transaction Script

(Not referring to a specific database technology)

```
Status TransferWithCheck (amount, account1, account2) {  
    funds = DB.read(account1);  
    if ( funds < amount ) {  
        Db.rollback();  
        return Status.insufficientFundsError;  
    } else {  
        DB.write (account1, funds-amount);  
        DB.write (account2, DB.read(account2)+amount);  
        DB.commit();  
        return Status.success;  
    }  
}
```

From the Tech News MongoDB



FOR GIANT IDEAS

SOLUTIONS



Eliot Horowitz

February 15, 2018



Category: [Company](#), [Technical](#)

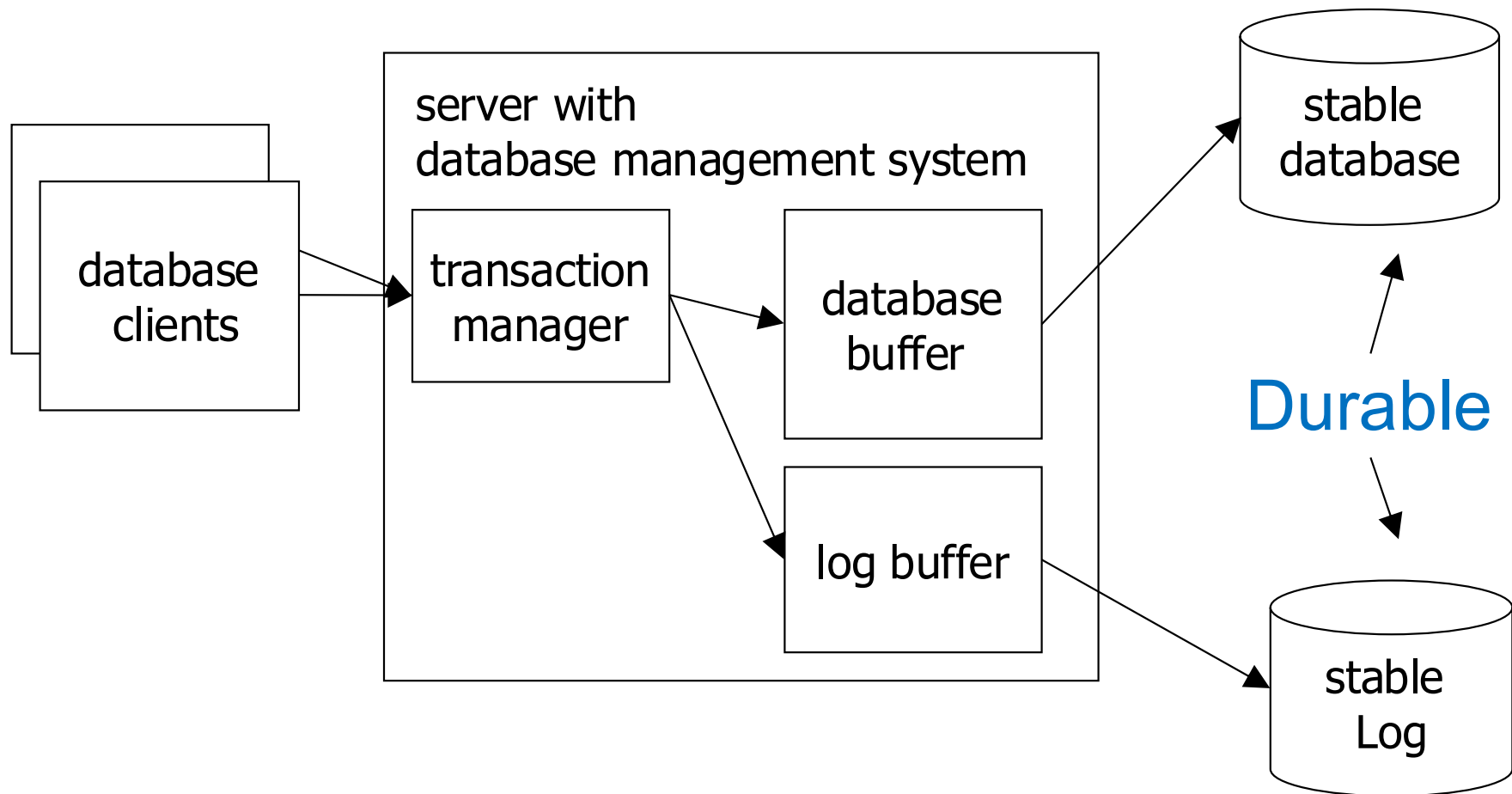
MongoDB 4.0 will add support for multi-document transactions, making it the only database to combine the speed, flexibility, and power of the document model with ACID data integrity guarantees. Through snapshot isolation, transactions provide a globally consistent view of data, and enforce all-or-nothing execution to maintain data integrity.

Transactions in MongoDB will feel just like transactions developers are familiar with from relational databases. They will be multi-statement, with similar syntax (e.g. `start_transaction` and `commit_transaction`), making them familiar to anyone with prior transaction experience. The changes to MongoDB that enable multi-document transactions will not impact performance for workloads that do not require them. In MongoDB 4.0, which will be released this summer*, transactions will work across a single

Atomicity requires rollback

- For Atomicity, the database must be able to undo write operations of uncommitted transactions.
- We consider here strategies that work with a **log**.
- The log is a list of records for write operations in the order of execution
 - optionally with some datastructures
 - for easier access.
- The log will be also important for recovery and ensuring Durability and will contain more information than we immediately need for rollback in the context of Atomicity.

Database internal architecture



The Database Log

- is a central feature of typical database managers.
- The log contains a list of
 - the write operations performed.
 - The transaction demarcations: (BOT), abort, commit.
- We consider undo/redo logs: for each write operation, a log entry is written that contains:
 - a log sequence number, or LSN (denoted as "nr:")
 - The transaction ID of the executing transaction ("ta:").
 - An identifier of the object affected ("obj:")
 - the value before the write operation (before-image, "b:").
 - the value after the write operation (after-image, "a:").

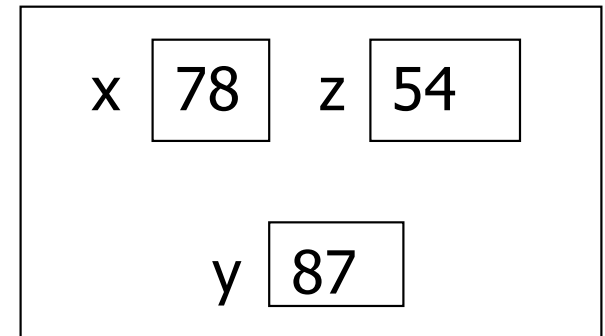
Example log and database

- The log of the database and the database buffer at a certain point in time could look the following way (new operations are appended at lower end)
- Transaction demarcations contain only LSN and TA.

.....

[nr: 109, ta: 22, BOT]
[nr: 110, ta: 22, obj: x, b: 2, a: 53]
[nr: 111, ta: 22, commit]
[nr: 112, ta: 23, BOT]
[nr: 113, ta: 23, obj: x, b: 53, a: 46]
[nr: 114, ta: 24, BOT]
[nr: 115, ta: 24, obj: y, b: 34, a: 87]
[nr: 116, ta: 23, obj: z, b: 23, a: 54]
[nr: 117, ta: 23, obj: x, b: 46, a: 78]

Log Sequence Number (LSN)



The state of the
buffer at this point
in time

Rollback

- A transaction that is aborted has to be undone in a **rollback**.

Rollback is the undoing of a transaction during normal operation (also known as transaction recovery, but we do not use this term).

- This rollback is based on the log.
 - An abort record is entered into the log.
- For all write operations performed so far
 - The before-image is restored.

Example: rollback of TA 23

- In case of a rollback, the operations are undone and the before-images are restored.

[nr: 109, ta: 22, BOT]

[nr: 110, ta: 22, obj: x, b: 2, a: 53]

[nr: 111, ta: 22, commit]

↪ [nr: 112, ta: 23, BOT]

↪ [nr: 113, ta: 23, obj: x, b: 53, a: 46]

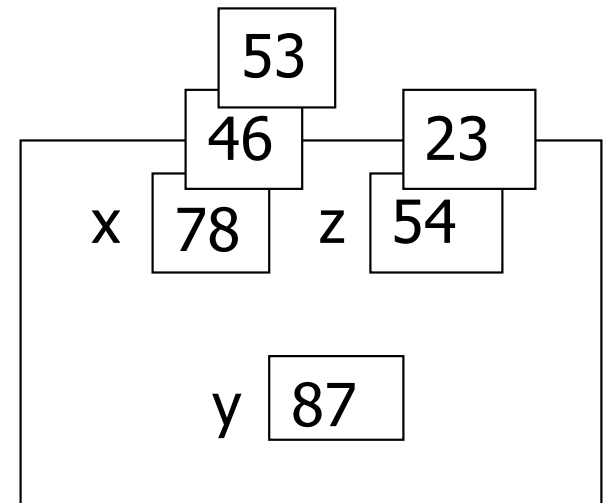
[nr: 114, ta: 24, BOT]

[nr: 115, ta: 24, obj: y, b: 34, a: 87]

↪ [nr: 116, ta: 23, obj: z, b: 23, a: 54]

↪ [nr: 117, ta: 23, obj: x, b: 46, a: 78]

↪ [nr: 118, ta: 23, abort]



Summary

1. we need transaction demarcation for atomicity:
for example for **remote** access.
2. **Transaction**: A sequence of operations that form a logical unit; the client has to request commit.
3. The database needs to roll back failed transactions.
4. The ACID Properties (Atomicity, Consistency, Isolation, Durability) are the requirements of transaction processing.
5. We consider Database Managers with undo/redo logs.